Wiley Interdisciplinary Reviews: Computational Molecular Science

WILEY

WIREs COMPUTATIONAL MOLECULAR SCIENCE

SOFTWARE FOCUS OPEN ACCESS

Software Update: The ORCA Program System—Version 6.0

Frank Neese

Max Planck Institut für Kohlenforschung, Mülheim an der Ruhr, Germany

Correspondence: Frank Neese (neese@kofo.mpg.de)

Received: 24 February 2025 | **Revised:** 19 March 2025 | **Accepted:** 20 March 2025

Associate Editor: Peter R. Schreiner | **Editor-in-Chief:** Peter R. Schreiner

Funding: This study was financially supported by the Max Planck Society. Very significant contributions to the ORCA 6.0 functionality were created by the employees of FACCTs (specifically Christoph Riplinger, Bernardo de Souza, Georgi Stoychev and Miquel Garcia-Rates) who are gratefully acknowledged for their dedication to keeping ORCA free of charge to the academic community.

Keywords: ab initio calculations | density functional theory | embedding methods | global optimization | quantum chemistry

ABSTRACT

Version 6.0 of the ORCA quantum chemistry program suite was released in July 2024. ORCA 6.0 is a major turning point in the history of the program since it represents a near complete rewrite of the code base that leads to: (1) major performance improvements, (2) a clean and highly efficient code base that greatly facilitates future development, (3) a large amount of new functionality, and (4) new interface capabilities that facilitate inter-operability with other quantum chemistry program packages. The article describes the most salient features of the program.

1 | Introduction

Developing a full-featured, general-purpose quantum chemistry package is a major undertaking that takes decades of dedicated effort to cover the main bases of the quantum chemical arsenal such as various flavors of Hartree-Fock (HF) and density functional theory (DFT), complete active space self-consistent field (CASSCF) methods followed by post-SCF methods such as second-order Møller-Plesset (MP2), coupled cluster (CC) theory or their multi-reference variants such as complete active space (CASPT2) or N-electron valence perturbation theory (NEVPT2), multi-reference (MR) configuration interaction (MR-CI) or even MR-CC. In order to be useful in chemical applications such programs also need to be able to optimize geometries, calculate first- and second-order properties such as electrical moments or magnetic resonance parameters, compute analytic harmonic frequencies (Hessians), treat electronic excited states as well as environment effects, feature embedding approaches for macromolecular or solid-state applications, and perhaps also allow for molecular dynamics (MD) calculations. Added complexity arises from combining any or all of the mentioned methods and computational tasks with an array of approximations such as the resolution of the identity (RI, also referred to as density fitting, DF), [1–3] the chain-of-spheres exchange (COSX), [4–6] Laplace transforms [7], or local approximations [8–11] to name only a few.

It is evident that such full-featured program packages are a result of the work of many individuals that will work on the code over the course of several decades. Almost invariably, a program package that grows to be (nearly) full-featured was not started with a master plan that would facilitate healthy growth over long periods of time. The almost inevitable consequence is, that such programs tend to become messy over time, are difficult to maintain, difficult to extend, and near impossible to adapt to the ever-evolving hardware landscape. The ultimate consequence of this evolution is that the programs will eventually be abandoned when incoming students refuse to work with the code or have to spend most of their time fighting the idiosyncrasies of the programming environment.

The above scenario is a typical course of evolution of large software projects and also pretty much describes the evolution of the ORCA package up to version 5.0 that was released in July of 2021 [12]. By that time, ORCA was already the second-most used quantum chemistry suite worldwide and had on the order of 35,000 academic users. The release of ORCA 5 marked an important landmark in the evolution of the program since it introduced the SHARK integral package that turned out to be highly performant and, in addition, introduced a new programming paradigm (the loop-kernel-consumer [LKC], concept) that led to highly compact and streamlined code [13]. At the time of release of ORCA 5.0, this new concept was implemented in a number of strategic places inside the existing ORCA infrastructure.

Following the release of ORCA 5, the user base grew very quickly and reached 80,000 in January of 2025. Quickly after the release of ORCA 5, it became evident that the new development concept is far-reaching and opened up a unique opportunity: the complete re-design of the code-base. This clearly is a daunting task, given that any major quantum chemistry program package consists of millions of lines of code. Nevertheless, the ORCA development team took on this challenge and managed to see it to completion, leading to the release of ORCA 6.0 in July of 2024. To the best of the authors knowledge, this was the first time ever, that a major quantum chemistry package was rewritten nearly from scratch. The result is a new, clean, lean, and efficient infrastructure that lends itself well to long-term evolution and healthy growth that can keep up with the developments in computer hardware. Thus, ORCA 6 can largely be regarded as a new quantum chemistry program, but perhaps the only one that was written with a specific master plan in mind that is the result of nearly three decades of experience in developing quantum chemical software.

The main part of the development was carried out by the teams at the Max Planck Institut für Kohlenforschung in Mülheim/Germany (https://orcaforum.kofo.mpg.de/app.php/portal) and at the company FAccTs (www.faccts.de) in Cologne, Germany, with excellent contributions from a number of international collaborators that are detailed in the release notes and the ORCA output. ORCA is distributed free of charge to academic users (https://orcaforum.kofo.mpg.de) and can be licensed for commercial use from FACCTs. The capabilities of the code are extensively documented in an online manual (https://www.faccts.de/docs/orca/6.0/manual/index.html) as well as a set of tutorials developed at FACCTs (https://www.orcasoftware.de/tutorials_orca/index.html).

As pointed out in previous versions of this text [12, 14, 15], highlights of ORCA include (1) State-of-the-art performance that was further significantly improved in ORCA 6, (2) comprehensive theoretical methodology, (3) user friendliness, and (4) platform independence. This short software update will provide an overview of the new infrastructure that ORCA 6 is based on as well as functionality added to ORCA since the publication of the 2021 update for ORCA 5.0 [12].

2 | The Architecture of ORCA 6

2.1 | Shell Structure

The overarching structure of ORCA 6.0 is shown in Figure 1. The leading idea behind the organization is a multilayer structure where each layer is communication with the layers above or below it via well-defined interfaces. This helps keeping the flow of logic and data coherent and allows developers to navigate a multi-million line software project. At the lowest-level of organization, the is the SHARK integral package that serves as a "motor" to the rest of the program. The SHARK package has a code base that makes nearly no reference to the ORCA inherent data structures or tools. It only uses a handful of tools, such as the ORCA inherent matrix/vector class and the associated BLAS and LAPACK routines, the tensor

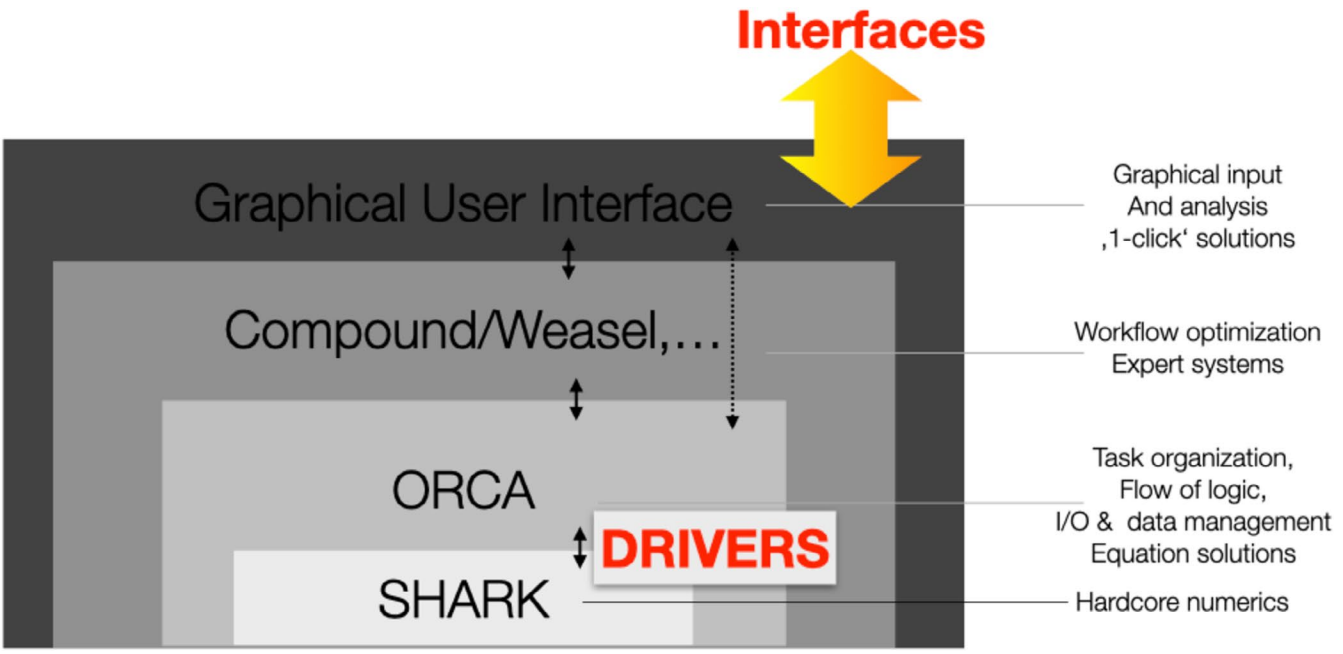

FIGURE 1 | Shell structure of the ORCA 6.0 code.

implementation and the "matrix-container" concept of ORCA. The rest of the SHARK infrastructure consists of streamlined and optimized routines that are organized in an object-oriented fashion. These objects are strongly structured and coherently named in the source code tree. SHARK performs the calculation of all integrals over atomic orbital (AO) basis functions and all integral transformations into the molecular orbital (MO). It also implements a host of functionality for the formation of Fock-like matrices featuring various approximations, such as the RI approximation.

The software layer above SHARK is the core ORCA program code. In ORCA 6.0, its main role is to organize the flow of logic through the entire program, the maintenance of intermediate data and the interaction with the user. While in the ORCA program code there are direct calls to SHARK routines, for the most part, compute intensive tasks are triggered by a reasonably small number of routines that we refer to as "DRIVERS." These driver routines go down into the "nitty-gritty" of how exactly a given Fock matrix or coupled-perturbed self-consistent field (CP-SCF) right hand side is computed. They determine what approximations (if any) are requested, whether these are available, whether solvent terms, external point charges or relativistic corrections are required, and so on. The benefit of such a structure is that a lot of rather intricate code could be removed from the core ORCA code and also does not burden the SHARK code base. For example, developers, who want to find out how a certain approximation to a given Fock matrix or integral transformation is being set up, only need to look in the DRIVER directory where a reasonable number of coherently named routines exist that coordinate the specific tasks at hand. Thus, in essence, the DRIVER infrastructure is highly helpful in keeping the ORCA and SHARK codes clean while providing a convenient and central place in the code base (that consists of more than 50,000 individual source code files) where developers find the required information together with a hands-on documentation.

Most computational studies consist of a significant number of individual steps that are carried out on, say, a collection of related molecules or on the same molecule with different computational settings, or perhaps a sequence of calculations is performed on one molecule in order to assemble an accurate total energy. Running these jobs individually is time-consuming and error-prone. To this end, many research groups have developed a collection of shell scripts that perform workflows. While this is a perfectly valid approach, it suffers from a few inefficiencies. For example, the transfer of the data from the program to the script may require parsing the output file, which may fail if the output of the program is changing. The person who has written the original script may not be available anymore, and the person wanting to use the script may not sufficiently understand a given script in order to adapt it. In ORCA 6.0, we offer a powerful functionality that is designed to help with such workflow problems. The "Compound" language is a simple scripting or programming language that can be used to automatize workflows. Its key advantage over external scripts is its deep integration with the ORCA data. Thus, a very large number of calculation results, such as total energies, excitation energies, molecular properties, geometries, or populations, are available readily inside a given compound script, and elementary algebraic operations as well as custom printouts can readily be created. On the commercial side, FACCTs offers a workflow tool (called Weasel) that creates professional workflow solutions that are often tailored to the specific needs of customers from industry.

Either Compound or Weasel communicates with ORCA via the "property" file. The property file contains a summary of the calculation results in a fixed human as well as machine readable format. In addition, ORCA 6.0 offers a JSON interface that is even more powerful and will be briefly described below.

The final layer of software consists of graphical user interfaces (GUI). Unfortunately, the academic ORCA development team does not have the resources to create a full-featured, professional GUI. Hence, so far, external GUIs, such as Avogadro [16], are frequently used to visualize ORCA results. The ORCA development team encourages GUI developers to interface their codes to ORCA, which should be greatly facilitated in ORCA 6.0 (and later) by the existence of the JSON [17] interface. A full-featured GUI will be developed, down the road, by FACCTs.

2.2 | Legacy Modules Deleted in ORCA 6

Since ORCA 6.0 is largely based on a new organizational concept and a largely overhauled code-base, there is a number of legacy modules that were deleted and replaced by new modules that were created from scratch. Most prominently, perhaps, the very central SCF module `orca_scf` fell victim to the update and is replaced by the two new modules `orca_guess` and `orca_leanscf` (described below) that perform the initial guess to the MOs and the actual SCF calculation, respectively. Second, the module `orca_cpscf` that was responsible for solving electric or magnetic response equations at the SCF level has been replaced with the far more streamlined, efficient, and powerful `orca_scfresp` that is part of a collection of response programs that collectively form the core of the ORCA 6 molecular property philosophy. This philosophy, discussed below, rendered the modules `orca_scfhess`, `orca_eprnmr`, and `orca_soc`, that previously calculated the SCF Hessian, EPR and NMR properties, and spin-orbit coupling integrals, respectively, superfluous. The module `orca_gtoint` is now more appropriately called `orca_startup` and has been redesigned completely. It coordinates initial tasks such as calculation of shell pair, the calculation of the one-electron matrices, set up of numerical integration grids, or the calculation and manipulation of RI-metric integrals among other tasks. Lastly, the module `orca_anoint`, that calculated integrals over atomic natural orbitals (ANOs) became superfluous since SHARK internally handles generally contracted basis set automatically as well as much more efficiently and comprehensively [13].

2.3 | The LeanSCF Program of ORCA 6

Pretty much every quantum chemistry package starts with the development of a SCF program. These codes are therefore usually the oldest in terms of program infrastructure. SCF programs also have the tendency to be the messiest part of a program package after generations of developers have

applied purpose-specific tweaks that may or may not be coherently implemented or documented. This was also the case for ORCA's old SCF program (`orca_scf`) that originated in 1999. Unfortunately, it became so difficult to navigate that it was nearly impossible to determine which data resides in the core memory of the computer at any given time. Since ORCA uses a replicated memory model for its parallelization strategy, the memory demands of the SCF program became a major bottleneck for performing very large SCF calculations with, say, 30,000–100,000 basis functions.

The flow of logic in `orca_scf` evolved to be so complicated that it became intransparent what quantities are calculated when and how. For example, in rewriting the code, the author found that the total energy could, under certain circumstances, be calculated up to four times during a given SCF cycle. This clearly happened because developers wanted "to be on the safe side" thus leading to inefficiencies of the indicated type. Even worse, some of these energy recalculations used the DIIS [18] extrapolated density thus leading to a non-variational energy.

For these reasons, it was decided to completely rewrite the SCF program with the main objectives of coherent, transparent and streamlined information flow and, most importantly, minimal memory demands. The new program, `orca_leanscf`, achieves these goals while being algorithmically equivalent, if not superior, to the previous `orca_scf` program. As memory savings matter, an infrastructure was created that allows the program to only have the minimal number of required matrices to be in main memory at any given time. For example, during the calculation of the density matrix, only the density **P** and the MO coefficient matrix **C** are in memory. Likewise, during the formation of the Fock matrix **F**, only **F** itself and **P** are in memory and during the orbital update only **F** and **C** are required. The non-required matrices are either deleted from memory ("Forget") or stored on disk if they have changed ("Sleep") while they can be brought back into memory with a single call ("WakeUp"). The time required for the input/output of these matrices remains negligible. The second memory saving measure is to hold matrices for which only read access is required (e.g., the metric matrix in RI calculations) in shared memory. Both measures significantly reduce the memory consumption of `orca_leanscf` and enable extremely large SCF calculations with over 10^4 and probably close to 10^5 basis functions that were previously far out of reach.

2.4 | Molecular Property Handling in ORCA 6

A major restructuring and streamlining with respect to molecular property calculations has taken place in ORCA 6.0. This was deemed desirable given the large number of electronic structure methods and modules in ORCA that in earlier program versions each had their own property branch. The result was not only an undesired duplication of code, but also inconsistencies arising from different flags and feature sets in different modules as well as a loss of efficiency in repeatedly recalculating the same property integrals. In ORCA 6, we differentiate between three principally different types of properties:

A. Molecular response properties that are formulated as derivatives of the ground state energy.

B. Quasi-degenerate perturbation theory (QDPT) properties that arise from the diagonalization of the QDPT matrix over a number of roots calculated by some method.

C. Excited state properties, for example, transition moments of various kinds connecting the electronic ground state with electronically excited states or electronically excited states with each other.

Each of the three property groups has a distinctive and streamlined infrastructure.

Turning first to the response properties, we notice that a first-order property, say λ, can always be calculated as:

$$\langle \lambda \rangle = \left.\frac{\partial E}{\partial \lambda}\right|_{\lambda=0} = \sum_{\mu\nu} P_{\mu\nu} \left\langle \mu \left| \frac{\partial \hat{H}}{\partial \lambda} \right| \nu \right\rangle \tag{1}$$

Here $P_{\mu\nu}$ is the density matrix in the AO basis $\{\mu\}$. The definition of the density matrix is method specific and varies between variational methods (such as Hartree-Fock, CASSCF or Full-CI) and non-or only partially variational methods (such as MP2, CCSD, or TD-DFT). It is nevertheless always possible to bring the first derivative of the energy with respect to a one-electron perturbation into a form consistent with Equation (1). A second-order (or response) property can be formulated as a second derivative of the energy as in:

$$\langle \lambda, \theta \rangle = \left.\frac{\partial^2 E}{\partial \lambda \partial \theta}\right|_{\lambda=\theta=0} = \sum_{\mu\nu} P_{\mu\nu} \left\langle \mu \left| \frac{\partial^2 \hat{H}}{\partial \lambda \partial \theta} \right| \nu \right\rangle + \sum_{\mu\nu} \frac{\partial P_{\mu\nu}}{\partial \theta} \left\langle \mu \left| \frac{\partial \hat{H}}{\partial \lambda} \right| \nu \right\rangle \tag{2}$$

The new quantities in Equation (2) are the second derivative of the Hamiltonian with respect to two perturbations and the first derivative of the density matrix with respect to one of the perturbations (response density). Given that the same formalism applies to all electric and magnetic perturbations as well as geometric derivatives, it is logical to treat the properties in one central module (`orca_prop`) and feed that module with method-specific densities, response densities, and property integrals.

With this philosophy in mind, the following sequence of steps is executed (see Figure 2):

1. Calculation of the energy of the electronic state in question and the one-particle density. The density is stored in a central place, the "density-container" that contains the actual density as well as the meta-data to uniquely define it.

2. Calculation of the necessary property integrals. To this end, the `orca` main program, that has oversight over the entire calculation, looks ahead and determines all types of integrals that are needed by any other module down the road. It then calls `orca_propint`, the program that calculates all necessary integrals over AOs and stores them in a central place, the "property integral container."

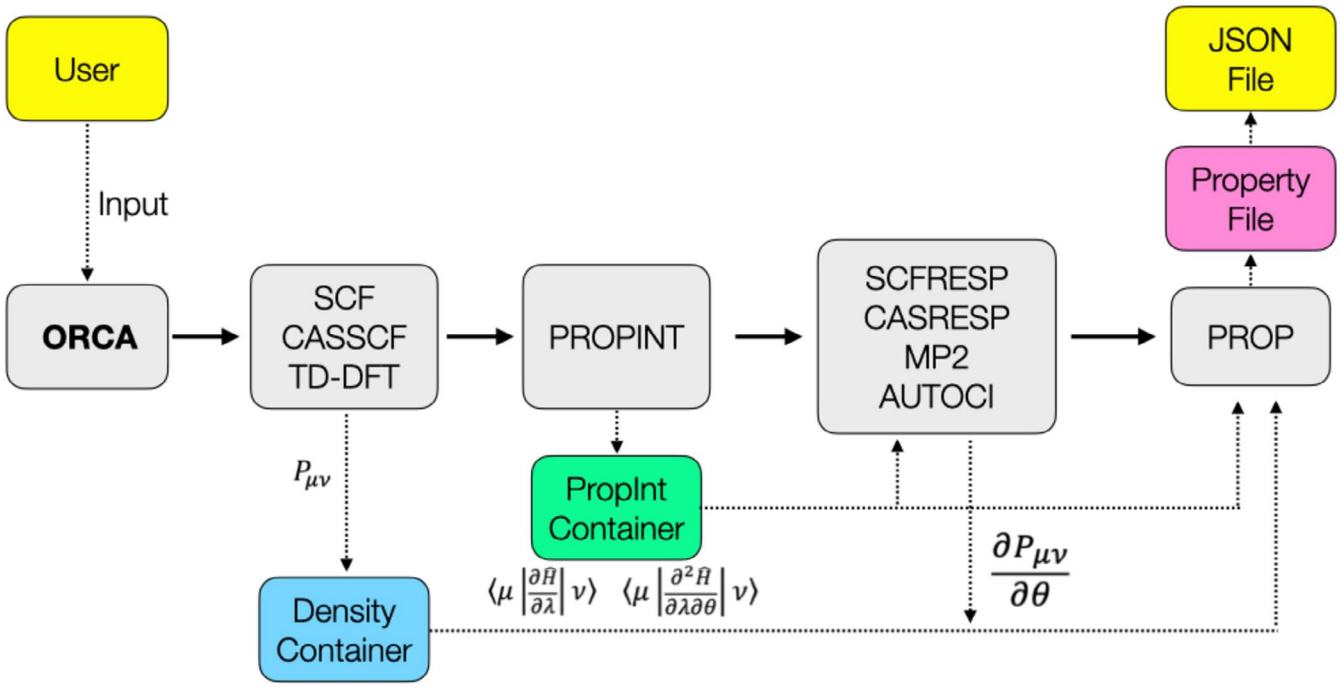

FIGURE 2 | Flow of information in the calculation of response properties in ORCA 6.0.

3. The property integrals are needed in order to calculate the response densities. These are calculated by separate modules, for example, `orca_scfresp` in the case of Hartree-Fock and DFT wavefunctions, `orca_casresp` for CASSCF, while other modules such as MP2 or AUTOCI can be run in "energy" or "response" mode in order to generate the equivalent data. The response densities are also stored in the density container.
4. The main program then calls `orca_prop`, which will determine which properties are requested and, for each requested property, will find and utilize all eligible densities, response densities, and property integrals. It will print the respective properties in a standardized format and store them in the property file that is available to the user.

This way of calculating properties is elegant and efficient and leads to a lean code structure since virtually no code is replicated while a large number of different method/property combinations are handled simultaneously. If a new electronic structure method is added to ORCA, all that is required is that it calculates the energy and the (response) density and all eligible properties will be available automatically without writing a single line of additional code. This structure also implies that the output may contain very useful method comparison data. For example, it is perfectly permissible to calculate the Hartree-Fock, MP2, and CCSD(T) energy and density in a single run. The output will then contain, say, the dipole moment calculated for all three methods in directly adjacent parts of the output, which facilitates comparison.

A similar concept is used in approaching QDPT and excited state properties. Without going into too many technical details, both of these property groups are handled consistently with a minimum of overhead using object-oriented tools that are as electronic structure method agnostic as possible.

Taken together, the new infrastructure strategy in ORCA has led to an enormous streamlining of the code that is not only much easier to read but has allowed us to delete hundreds of thousands of lines of difficult-to-maintain legacy code. One benefit of the new infrastructure was, for example, the implementation of vibrational circular dichroism (VCD) intensities. This is a rather complex property that contains electric, magnetic, and geometric perturbations [17, 19]. The new infrastructure allowed for an implementation of VCD in about 1 day of work (admittedly followed by a few days of tweaking and debugging).

2.5 | Efficiency Improvements

Together with the significant memory streamlining of `orca_leanscf`, there is a substantial number of efficiency improvements in ORCA's algorithms. They are too technical and too numerous to describe in detail in this short overview article, but a few points might be noted:

a. The chain-of-spheres (COSX) algorithm has been significantly tweaked and runs a factor of 2–4 faster than in ORCA 5 while keeping the same or even superior accuracy. This leads to enormous speedups in hybrid DFT and Hartree-Fock calculations.
b. The Split-RI-J algorithm is up to 20% faster than before [4] and now outperforms the original algorithm of 2003 while being applicable to high-angular momentum functions (before the limit was $L = 4$) and groups of densities in a particularly efficient manner.

c. The SCF convergers are more robust now and on average take about 20% fewer cycles than before to reach convergence.

d. The SCF stability analysis has been completely reworked and now features all approximations that ORCA is capable of and, also unlike the previous implementation, allows for solvent effects to be present. Finding a stable solution has been automatized.

e. The geometry optimizer has been improved in many significant details. It is more robust than previously and also far less prone to end up in geometries with small imaginary frequencies (that usually belong to methyl group rotations and similar low energy conformational motions).

f. The nudge-elastic-band (NEB) [20] transition state finder has been improved in a number of significant details and will now find transition states in fewer numbers of iterations.

g. The efficiency of SCF Hessian calculation has been significantly improved in a number of ways. First of all, the efficiency of the integral calculations is generally higher than in ORCA 5. Second, the solution of the coupled-perturbed SCF (CP-SCF) equations has been improved significantly by the concept of "group parallelization". In this concept, the available cores are divided into groups that each work on a subset of geometric displacements. Since the groups are independent of each other and inside each group (4–8 processes) the parallel scaling is close to perfect, this leads to large speedups in Hessian calculations on large systems.

h. The calculation of the so-called coupling coefficients for solving large configuration interaction problems in CASSCF calculations has been drastically improved [21] and now takes a fraction of a second for a large number of open-shells, while previously the same task might have taken hours.

2.6 | Workflow Streamlining and the JSON Interface to ORCA 6

A further focus point of ORCA 6 development was to streamline workflow handling of complex computational tasks and to make it drastically easier to integrate ORCA into external workflow solutions.

The compound scripting language has been significantly extended and improved. It features elementary programming constructs such as variable declarations and algebra with variables as well as elementary programming structures such as "for," "if/else," and "while." Importantly, scripts can be written in a rather general manner because once they are called from a given ORCA input file, the values of the variables in the script can be modified using the "with" command. Likewise, in each compound step (that corresponds to one ORCA input file) generic variables can be introduced by the "&{}" command, which is very useful for running calculations in which certain computational settings are being permuted. A fully functional example for a Compound script that evaluates the basis set limit SCF energy by extrapolation is shown in Figure 3.

The important aspect of compound jobs over external shell scripts, however, is the deep integration with the ORCA calculation results that can be accessed following each individual step using the "read" command. In this way, complex workflows can easily be implemented, or large sets of benchmark calculations can be run with high confidence and minimal effort. Using compound infrastructure, a large number of advanced calculations can be realized that were not possible before; for example, the optimization of geometric structures using basis set extrapolated energies and gradients, or the optimization of molecular clusters with taking into account the basis-set-superposition error (BSSE) correction.

In order to provide a transparent interface to ORCA that eliminates the need for external tools and programs to parse the output file (something that the ORCA team will no longer support), we have created the property file and the JSON [17] interface. The JSON format was chosen because it represents an industry standard for data exchange and a large amount of programming tools exist to process files written in JSON format.

The property file is an ASCII file that is human and machine readable and contains a concise summary of the various properties that were calculated during the ORCA run, including energies, energy components, geometries, molecular properties, and so on. This file can be translated to JSON format using the `orca_2json` program. However, the JSON interface can do much more than that. Using a user-provided configuration file, the `orca_2json` program can be used to make ORCA print geometries, basis sets, and a large variety of integrals over atomic or MOs, and it can also be used to generate GBW files that can be used to feed ORCA. Thus, with this interface, users can use ORCA as an integral generator for their own programs, as an external geometry optimizer to be used with their energies and gradients, or users can feed ORCA with their own orbitals. This functionality will define the official ORCA interface, which, from now on, is the *only* interface that the ORCA team will support, extend, and maintain.

Finally, a citation tool has been implemented that collects suggested citations at the end of a given ORCA run and also leaves a bibtex formatted file behind, which should help users to correctly cite ORCA relevant references in their publications and lower the barrier to do so.

3 | New Functionality in ORCA 6

Since the length of this article does not allow a careful discussion of any individual subject, this section will provide a brief, almost bullet-point-like, documentation of new features in ORCA 6.

3.1 | SCF and Single Reference Correlation Methods

ORCA 6 now features the Delta-SCF method [22, 23]. Delta-SCF allows the user to converge the SCF equation on nearly arbitrary occupations. This can be used, for example, for the calculation of excited states including electronic relaxation or the calculation of

```
%compound
  Variable Basis      = {"cc-pVDZ", "cc-pVTZ", "cc-pVQZ", "cc-pV5Z", "cc-pV6Z"};
  Variable Cardinal   = {    2    ,     3    ,     4    ,     5    ,     6    };
  Variable alpha      = {    4.42 ,     5.46 ,     9.19 ,     9.19,      9.19 };
  Variable Eh2kcal    = 27.2107*23.06;
  Variable EN[Basis.GetSize()];
  Variable ibas;
  for ibas from 0 to Basis.GetSize()-1 do
    New_Step
    ! RHF &{Basis[ibas]}  VeryTightSCF Conv
    Step_end
    alias currStep;
    read EN[ibas] = JOB_INFO_TOTAL_EN[currStep];
 endfor;

 Variable Extrap[Basis.GetSize()];
 Variable EX,EY,ExpX,ExpY,X,Y;
 for ibas from 0 to Basis.GetSize()-2  do
   EX       = EN[ibas];
   EY       = EN[ibas+1];
   X        = Cardinal[ibas];
   Y        = Cardinal[ibas+1];
   ExpX     = exp(-alpha[ibas]*sqrt(X));
   ExpY     = exp(-alpha[ibas]*sqrt(Y));
   Extrap[ibas] = (EX*expY-EY*expX)/(ExpY-ExpX);
 endfor;

 Variable Eref = Extrap[Basis.GetSize()-2];
 print("Reference energy           = %16.12lf\n",Eref);
 for ibas from 0 to Basis.GetSize()-1 do
   print("Cardinal number %d : Energy = %16.12lf Error= %9.6lf Eh = %9.2lf kcal/mol\n",
         Cardinal[ibas],EN[ibas],EN[ibas]-Eref,Eh2kcal*(EN[ibas]-Eref));
 endfor;
 for ibas from 0 to Basis.GetSize()-2 do
   print("Extrapolation %d/%d : Energy = %16.12lf Error= %9.6lf Eh = %9.2lf kcal/mol\n",
         Cardinal[ibas],Cardinal[ibas+1],Extrap[ibas],Extrap[ibas]-Eref,Eh2kcal*(Extrap[ibas]-Eref));
 endfor;
endrun;
```

FIGURE 3 | Example of a compound script that Calculates the basis set limit SCF energy by extrapolation and evaluates the error relative to the CBS energy for calculations with and without extrapolation. The script demonstrates the deep integration with ORCA via "Read," custom algebra, and custom printouts as well as simple "for" loops.

electronic coupling matrix elements in electron-transfer studies, to name only two out of many applications.

The ROHF capabilities of ORCA have been extended by a very useful method that allows users to converge the SCF equations on an arbitrary, user-defined configuration state function (General CSF-ROHF) [24]. This is very useful for the calculation of anti-ferromagnetic coupled systems, transition metal clusters, and similar systems.

While ORCA always had the (undocumented) ability to perform SCF calculations in the presence of external electric fields, this feature is now officially supported and has also been extended to allow for geometry optimizations, transition state calculations, and so on.

The list of DFT functionals supported by ORCA has been extended and includes the latest developments reported by the Grimme group, for example, functionals of the SCAN family and new range-separated functionals [25–27].

The coupled-cluster module of ORCA (`orca_mdci`) has been extended to allow for STEOM-CCSD calculations on open-shell systems [28, 29]. The new implementation is far more efficient than the original one reported earlier [30].

ORCA 6 allows for full-featured cluster-in-molecules local correlation calculations using MP2, CCSD(T) and their DLPNO- and also explicitly correlated (F12) variants to be used for the individual fragments [31, 32]. Since these calculations involve fragment calculations on individual subsystems of the whole system, parallelization efficiency is high and memory consumption is lower than for, say, DLPNO calculations on the whole system.

3.2 | CASSCF

In addition to the introduction of the CASSCF linear response program mentioned above, the CASSCF program largely profits from the reworked infrastructure and runs more efficiently and reliably with better convergence behavior than previously. A significant extension has been the availability of the Automated Construction of Molecular Active Spaces from Atomic Valence Orbitals (AVAS) [33] guess, which allows users to converge to the desired active spaces in a transparent and controlled manner. ORCA 6 now also features a quadratically converging second-order converger based on the trust-radius augmented Hessian (TRAH) approach [34] Finally, as mentioned above, the drastically improved calculation of coupling coefficients greatly speeds up the CI step for larger.

3.3 | Automatic Code Generation for Correlation Methods

A major amount of work has gone into the improvement of the functionality of the automatically generated code for correlation calculations in the framework of the AUTOCI module. The infrastructure has been massively improved, leading to efficient code that frequently rivals the best hand written code available in ORCA or elsewhere [35]. AUTOCI is also parallelized, although the efficiency is somewhat moderate given that AUTOCI runs require a significant amount of input and output, which interferes with parallel scaling to some extent. The speedup is, however, reasonable up to about 8–16 cores.

Based on the efforts that went into AUTOCI, it now features analytic gradients for coupled cluster methods including CCSD and CCSD(T) with or without frozen cores for RHF and UHF reference wavefunctions. In addition, there is a coupled cluster linear response module that allows for response property calculations like polarizabilities. In addition to coupled cluster methods up to and including CCSDT, ORCA 6 also features Møller-Plesset perturbation theory through fifth order for RHF and UHF references including analytic gradients and relaxed density matrices.

AUTOCI has been optimized to the point where fairly large calculations are feasible. We estimate the upper limits to be about 1000 basis functions for CCSD/MP4(SD), 600–800 basis functions for CCSD gradients, and CCSD(T)/MP4. Further optimization will certainly prove possible, and it is worth emphasizing that due to the "deep integration" approach that we have taken [35] any improvement in the generation chain will automatically benefit all automatically generated code.

3.4 | Relativity

A very significant improvement in ORCA 6 is the implementation of the X2C relativistic Hamiltonian [36, 37]. As is customary with ORCA, X2C is featured in a scalar relativistic variant and features first- and second-order picture change corrections as well as analytic gradients. Hessians are calculated semi-numerically from analytic gradients.

3.5 | Excited States

ORCA features a significant number of methods to calculate electronically excited states including MRCI, CIS/TD-DFT (RPA or RDA), ROCIS, NEVPT2, and CASPT2, MC-RPA among others. All of these methods can now be combined with the new "one-photon absorption" (OPA) infrastructure that allows for the calculation of transition moments using the full, exact field–matter interaction Hamiltonian [38].

ORCA 6 also features the general-spin ROCIS (GS-ROCIS) method that allows for ROCIS calculations on top of the general CSF-ROHF reference wavefunctions. This comes in very handy in the calculation of the spectra of antiferromagnetically coupled molecules or antiferromagnetic surfaces [39].

In addition, new spectroscopic properties are available including vibrationally resolved CD and MCD spectra as well as new approaches for the calculation of Fluorescence and Phosphorescence spectra as well as radiationless decay rates.

The VPT2 module in ORCA has also been significantly extended for the calculation of vibrationally averaged molecular properties such as NMR chemical shifts, spin–spin coupling constants, g-tensors, and hyperfine couplings among many other technical improvements.

3.6 | Explicit and Implicit Solvation and Embedding

Many technical improvements have been made to the conductor like polarizable continuum (CPCM) based implementation in ORCA, including an improved isodensity scheme for the surface meshes used, much higher efficiency in the integral calculation through shark, the "PTES" scheme for DLPNO-CCSD(T) calculation in the presence of solvent, and the SMD analytic hessian, among other improvements [40].

ORCA 6 features three very powerful new tools that are aimed at dealing with conformational complexity. First and foremost, the GOAT algorithm is a global optimizer that is able to identify families of low-lying conformers in complex systems [41]. The tool named DOCKER allows for optimal interaction geometries between a pair of interacting molecules to be found automatically. The tool called SOLVATOR is an automatized and streamlined way to create explicit solvation shells to a given solute.

ORCA 6 also features the fast-multipole-method (FMM) for the linear scaling calculation of the point charge contributions in QM/MM and electrostatic embedding calculations [42]. It leads to major speedups in calculations with very many (hundreds-of-thousands to millions) of point charges.

4 | Conclusion

As mentioned in the introduction, ORCA 6 is not primarily an update or a tweak to an existing program but essentially a completely reworked and greatly streamlined suite of quantum chemistry programs. This was a long-term effort by the development team that started in the spring of 2020 and came to its present culmination point with the release of ORCA 6.0 in July of 2024. Probably around 90% of the core infrastructure of ORCA has been rewritten and a completely new flow of logic has been implemented. These improvements have greatly added to the efficiency of ORCA 6.0 that runs significantly faster than any previous version of the program. While there are still some remaining streamlining tasks, ORCA 6.0 now provides a very powerful and convenient tool for efficient further development. This will manifest itself in shorter release cycles and an increased number of efficiently implemented features in each release. At the same time, it has been an important objective of the reworked infrastructure to facilitate the integration of ORCA into workflow solutions. To this end, significant effort was invested in the JSON interface that allows researchers to

interface ORCA, for example for integration into a GUI, to use ORCA as an integral generator or external optimizer or for connection to downstream analysis tools to name only a few areas of application.

Creating ORCA 6.0 was a tour-de-force and a labor of love and great dedication. The ORCA team is proud to release this version of the code to the general public.

Author Contributions

Frank Neese: conceptualization (equal), data curation (equal), formal analysis (equal), funding acquisition (equal), investigation (equal), methodology (equal), project administration (equal), software (equal), supervision (equal), validation (equal), visualization (equal), writing – original draft (equal), writing – review and editing (equal).

Acknowledgments

The creation of ORCA 6.0 was a monumental effort that required a level of dedication that is probably only possible once in a lifetime. That the team managed to collectively push through the multiple challenges met while chasing the seemingly ever-evading finish line was a humbling, memorable, and highly encouraging experience. I am deeply indebted to everybody who put their hard work and their enthusiasm into this project. Open Access funding enabled and organized by Projekt DEAL.

Conflicts of Interest

The author is co-founder of the company FACCTs (https://www.faccts.de), which is commercially distributing ORCA to industry (ORCA is and will remain free for academic researchers). The authors declare no conflicts of interest.

Data Availability Statement

Data sharing is not applicable to this article as no new data were created or analyzed in this study.

Related WIREs Articles

Software update: the ORCA program system, version 4.0

Software update: The ORCA program system-Version 5.0

The ORCA program system

References

1. E. J. Baerends, D. E. Ellis, and P. Ros, "Self-Consistent Molecular Hartree-Fock-Slater Calculations—I. The Computational Procedure," *Chemical Physics* 2, no. 1 (1973): 41–51.

2. O. Vahtras, J. Almlöf, and M. W. Feyereisen, "Integral Approximations for LCAO-SCF Calculations," *Chemical Physics Letters* 213, no. 5–6 (1993): 514–518.

3. J. L. Whitten, "Coulombic Potential Energy Integrals and Approximations," *Journal of Chemical Physics* 58 (1973): 4496.

4. B. Helmich-Paris, B. de Souza, F. Neese, and R. Izsak, "An Improved Chain of Spheres for Exchange Algorithm," *Journal of Chemical Physics* 155, no. 10 (2021): 104109.

5. R. Izsak and F. Neese, "An Overlap Fitted Chain of Spheres Exchange Method," *Journal of Chemical Physics* 135, no. 14 (2011): 144105.

6. F. Neese, F. Wennmohs, A. Hansen, and U. Becker, "Efficient, Approximate and Parallel Hartree-Fock and Hybrid DFT Calculations. A 'Chain-of-Spheres' Algorithm for the Hartree-Fock Exchange," *Chemical Physics* 356, no. 1–3 (2009): 98–109.

7. J. Almlöf, "Elimination of Energy Denominators in Møller-Plesset Perturbation-Theory by a Laplace Transform Approach," *Chemical Physics Letters* 181, no. 4 (1991): 319–320.

8. C. Hampel and H. J. Werner, "Local Treatment of Electron Correlation in Coupled Cluster Theory," *Journal of Chemical Physics* 104, no. 16 (1996): 6286–6297.

9. F. Neese, A. Hansen, and D. G. Liakos, "Efficient and Accurate Approximations to the Local Coupled Cluster Singles Doubles Method Using a Truncated Pair Natural Orbital Basis," *Journal of Chemical Physics* 131, no. 6 (2009): 15.

10. F. Neese, F. Wennmohs, and A. Hansen, "Efficient and Accurate Local Approximations to Coupled-Electron Pair Approaches: An Attempt to Revive the Pair Natural Orbital Method," *Journal of Chemical Physics* 130, no. 11 (2009): 18.

11. P. Pulay, "Localizability of Dynamic Electron Correlation," *Chemical Physics Letters* 100, no. 2 (1983): 151–154.

12. F. Neese, "Software Update: The ORCA Program System-Version 5.0," *Wiley Interdiscip Reviews: Computional Molecular Science* 12 (2022): 15.

13. F. Neese, "The SHARK Integral Generation and Digestion System," *Journal of Computational Chemistry* 44 (2022): 381–396.

14. F. Neese, "The ORCA Program System," *WIREs Computational Molecular Science* 2, no. 1 (2012): 73–78.

15. F. Neese, "Software Update: The ORCA Program System, Version 4.0," *Wiley Interdiscip Reviews: Computional Molecular Science* 8, no. 1 (2018): e1327.

16. M. D. Hanwell, D. E. Curtis, D. C. Lonie, T. Vandermeersch, E. Zurek, and G. R. Hutchison, "Avogadro: An Advanced Semantic Chemical Editor, Visualization, and Analysis Platform," *Journal of Cheminformatics* 12 (2012): 4–17.

17. F. Pezoa, J. L. Reutter, F. Suarez, M. Ugarte, and D. Vrgoc, "Foundations of JSON Schema," in *25th International World Wide Web Conferences Steering Committee* (Republic and Canton of Geneva, 2016).

18. P. Pulay, "Convergence Acceleration of Iterative Sequences—The Case of Scf Iteration," *Chemical Physics Letters* 73, no. 2 (1980): 393–398.

19. K. L. Bak, F. J. Devlin, C. S. Ashvar, P. R. Taylor, M. J. Frisch, and P. J. Stephens, "Ab Initio Calculation of Vibrational Circular Doichroism Spectra Using Gauge-Invariant Atomic Orbitals," *Journal of Physical Chemistry* 99 (1995): 14918–14922.

20. V. Asgeirsson, B. O. Birgisson, R. Bjornsson, et al., "Nudged Elastic Band Method for Molecular Reactions Using Energy-Weighted Springs Combined With Eigenvector Following," *Journal of Chemical Theory and Computation* 17, no. 8 (2021): 4929–4945.

21. M. Ugandi and M. Römelt, "A Recursive Formulation of One-Electron Coupling Coefficients for Spin-Adapted Configuration Interaction Calculations Featuring Many Unpaired Electrons," *International Journal of Quantum Chemistry* 123 (2023): e27045.

22. Y. L. A. Schmerwitz, G. Levi, and H. Jónsson, "Calculations of Excited Electronic States by Converging on Saddle Points Using Generalized Mode Following," *Journal of Chemical Theory and Computation* 19, no. 12 (2023): 3634–3651.

23. E. Selenius, A. E. Sigurdarson, Y. L. A. Schmerwitz, and G. Levi, "Orbital-Optimized Versus Time-Dependent Density Functional Calculations of Intramolecular Charge Transfer Excited States," *Journal of Chemical Theory and Computation* 20, no. 9 (2024): 3809–3822.

24. T. L. D. Gouveia, D. Maganas, and F. Neese, "Restricted Open-Shell Hartree-Fock Method for a General Configuration State Function

Featuring Arbitrarily Complex Spin-Couplings," *Journal of Physical Chemistry. A* 128, no. 25 (2024): 5041–5053.

25. M. Bursch, H. Neugebauer, S. Ehlert, and S. Grimme, "Dispersion Corrected r^{2}SCAN Based Global Hybrid Functionals: r^{2}SCANh, r^{2}SCAN0, and r^{2}SCAN50," *Journal of Chemical Physics* 156, no. 13 (2022): 134105.

26. S. Grimme, A. Hansen, S. Ehlert, and J. M. Mewes, "r^{2}SCAN-3c: A 'Swiss Army Knife' Composite Electronic-Structure Method," *Journal of Chemical Physics* 154, no. 6 (2021): 064103.

27. L. Wittmann, H. Neugebauer, S. Grimme, and M. Bursch, "Dispersion-Corrected r^{2}SCAN Based Double-Hybrid Functionals," *Journal of Chemical Physics* 159, no. 22 (2023): 224103.

28. M. Casanova-Páez and F. Neese, "Assessment of the Similarity-Transformed Equation of Motion (STEOM) for Open-Shell Organic and Transition Metal Molecules," *Journal of Chemical Physics* 161, no. 14 (2024): 1306–1321.

29. M. Casanova-Páez and F. Neese, "Core-Excited States for Open-Shell Systems in Similarity-Transformed Equation-of-Motion Theory," *Journal of Chemical Theory and Computation* 21, no. 3 (2025): 1306–1321.

30. L. M. J. Huntington, M. Krupicka, F. Neese, and R. Izsák, "Similarity Transformed Equation of Motion Coupled-Cluster Theory Based on an Unrestricted Hartree-Fock Reference for Applications to High-Spin Open-Shell Systems," *Journal of Chemical Physics* 147, no. 17 (2017): 174104.

31. Y. Q. Wang, Y. Guo, F. Neese, E. F. Valeev, W. Li, and S. H. Li, "Cluster-in-Molecule Approach With Explicitly Correlated Methods for Large Molecules," *Journal of Chemical Theory and Computation* 19, no. 22 (2023): 8076–8089.

32. Y. Q. Wang, Z. G. Ni, F. Neese, W. Li, Y. Guo, and S. H. Li, "Cluster-in-Molecule Method Combined With the Domain-Based Local Pair Natural Orbital Approach for Electron Correlation Calculations of Periodic Systems," *Journal of Chemical Theory and Computation* (2022): 1549–9626.

33. E. R. Sayfutyarova, Q. Sun, G. K. L. Chan, and G. Knizia, "Automated Construction of Molecular Active Spaces From Atomic Valence Orbitals," *Journal of Chemical Theory and Computation* 13 (2017): 4063–4078.

34. B. Helmich-Paris, "A Trust-Region Augmented Hessian Implementation for State-Specific and State-Averaged CASSCF Wave Functions," *Journal of Chemical Physics* 156 (2022): e204104.

35. M. H. Lechner, A. Papadopoulos, K. Sivalingam, et al., "Code Generation in ORCA: Progress, Efficiency and Tight Integration," *Physical Chemistry Chemical Physics* 26, no. 21 (2024): 15205–15220.

36. W. Kutzelnigg and W. Liu, "Quasirelativistic Theory Equivalent to Fully Relativistic Theory," *Journal of Chemical Physics* 123 (2005): 241102.

37. H. J. A. Jensen, W. Kutzelnigg, W. Liu, T. Saue, and L. Visscher, "The Generic Acronym 'X2C' for Exact Two-Component Hamiltonians Resulted From Intensive Discussions Among," in *During the Twelfth International Conference on the Applications of Density Functional Theory DFT-2007* (2007), 26–30.

38. N. Foglia, B. De Souza, D. Maganas, and F. Neese, "Including Vibrational Effects in Magnetic Circular Dichroism Spectrum Calculations in the Framework of Excited State Dynamics," *Journal of Chemical Physics* 158, no. 15 (2023): 154108.

39. T. L. D. Gouveia, D. Maganas, and F. Neese, "General Spin-Restricted Open-Shell Configuration Interaction Approach: Application to Metal K-Edge X-Ray Absorption Spectra of Ferro- and Antiferromagnetically Coupled Dimers," *Journal of Physical Chemistry. A* 129, no. 1 (2024): 330–345.

40. M. Garcia-Rates, U. Becker, and F. Neese, "Implicit Solvation in Domain Based Pair Natural Orbital Coupled Cluster (DLPNO-CCSD) Theory," *Journal of Computational Chemistry* 42, no. 27 (2021): 1959–1973.

41. B. de Souza, "GOAT: A Global Optimization Algorithm for Molecules and Atomic Clusters," *Angewandte Chemie* (2025): e202500393, https://doi.org/10.1002/anie.202500393 (in press).

42. P. Colinet, F. Neese, and B. Helmich-Paris, "Improving the Efficiency of Electrostatic Embedding Using the Fast Multipole Method," *Journal of Computational Chemistry* 46, no. 1 (2025): e27532.

Bibliography

1. For further information, the reader is encouraged to visit https://orcaforum.kofo.mpg.de, https://www.orcasoftware.de/tutorials_orca/index.html, the ORCA 6 online manual https://www.faccts.de/docs/orca/6.0/manual/index.html and the ORCA (https://www.youtube.com/@orcaquantumchemistry/featured) as well as FACCTs (https://www.youtube.com/@faccts_orca/featured) YouTube channels. These sites involve an active discussion forum, step-by-step tutorials as well as video lectures covering many of the subjects discussed here.